All DML statements can benefit from batching as the following tests demonstrate. Just like for the JDBC `Statement` test case, the same amount of data (1000 *post* and 4000 *comments*) is inserted, updated, and deleted while varying the batch size.

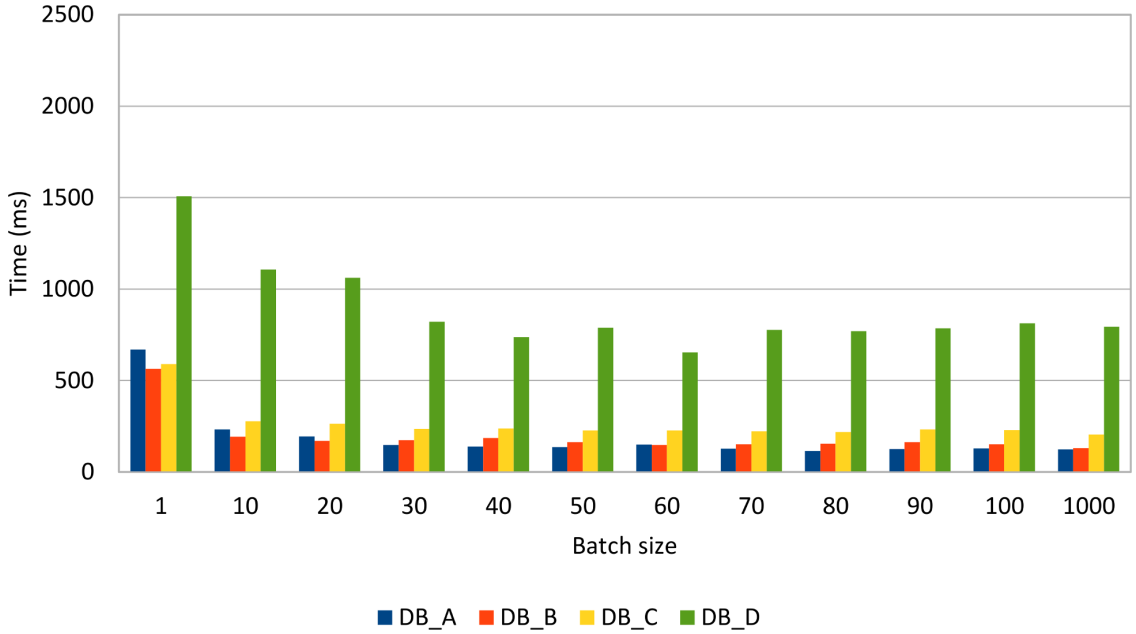

Figure 3.4: Insert PreparedStatement batch size

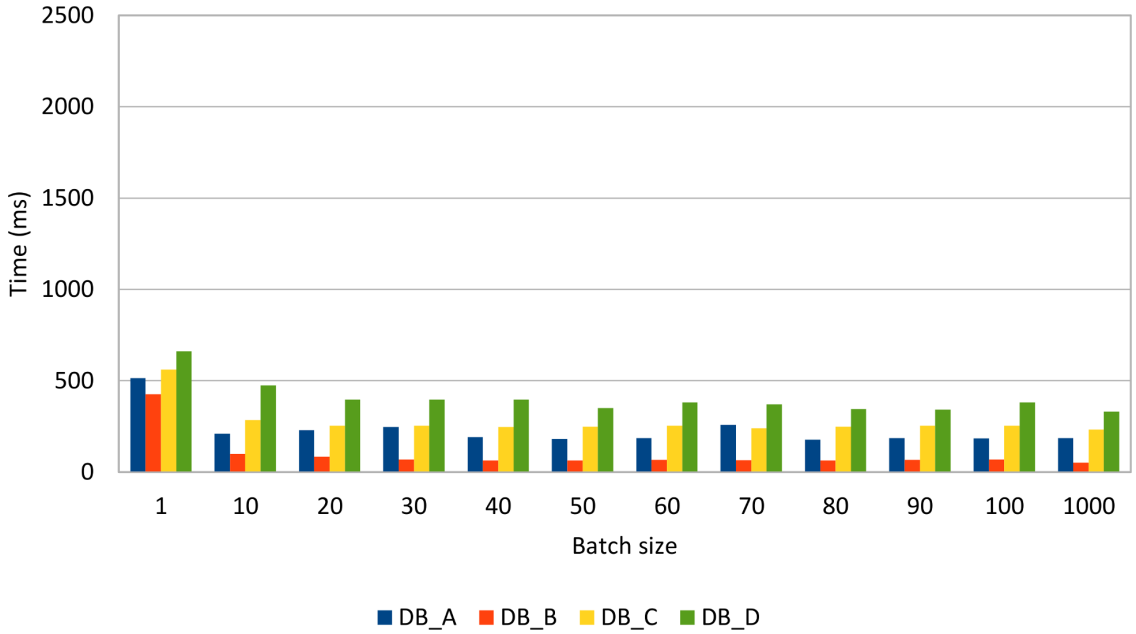

Figure 3.5: Update PreparedStatement batch size

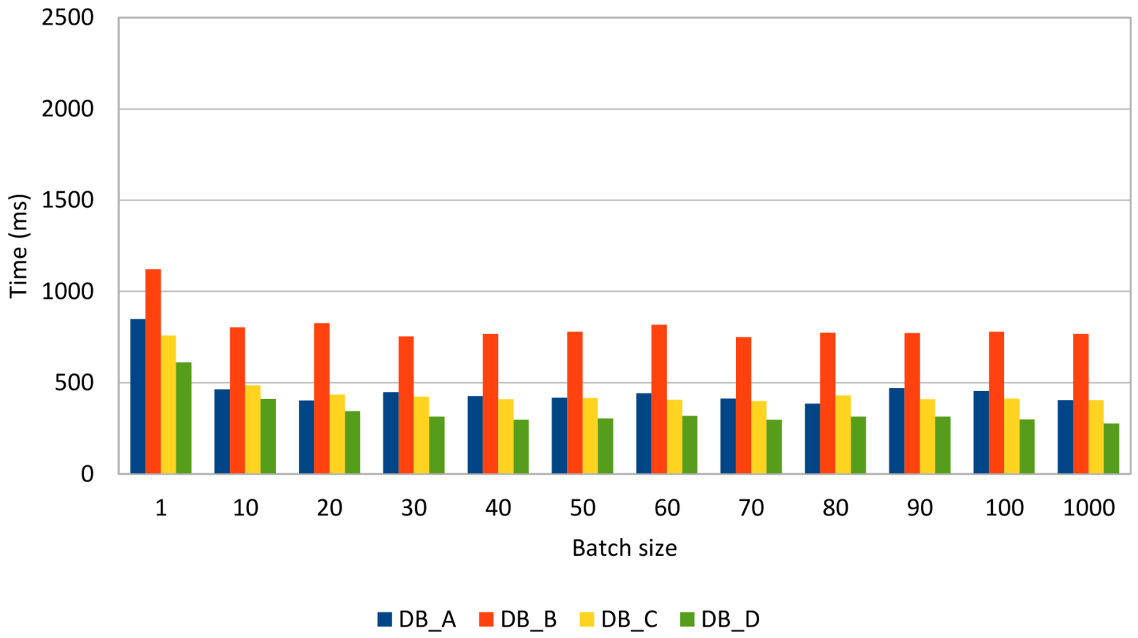

Figure 3.6: Delete PreparedStatement batch size

All database systems show a significant performance improvement when batching prepared statements. Some database systems are very fast when inserting or updating rows while others perform very well when deleting data.

Compared to the previous `Statement` batch insert results, it is clear that, for the same data load, the `PreparedStatement` use case performs just better. In fact, `Statement(s)` should not be used for batching CRUD operations, being more suitable for bulk processing:

```
DELETE from post
WHERE spam = true AND created_on < current_timestamp - INTERVAL '30' day;
```

3.2.1 Choosing the right batch size

Finding the right batch size is not a trivial thing to do as there is no mathematical equation to solve the appropriate batch size for any enterprise application.

Like any other performance optimization technique, measuring the application performance gain in response to a certain batch size value remains the most reliable tuning option.

The astute reader has already figured out that even a low batch size can reduce the transaction response time, and the performance gain does not grow linearly with batch size. Although a larger batch value can save more database roundtrips, the overall performance gain does not necessarily increase linearly. In fact, a very large batch size can hurt application performance if the transaction takes too long to be executed.

As a rule of thumb, you should always measure the performance improvement for various batch sizes. In practice, a relatively low value (between 10 and 30) is usually a good choice.

3.2.2 Bulk processing

Apart from batching, SQL offers bulk operations to modify all rows that satisfy a given filtering criteria. *Bulk update* or *delete* statements can also benefit from indexing, just like select statements.

To update all records from the previous example, one would have to execute the following statements:

```
UPDATE post SET version = version + 1;
UPDATE post_comment SET version = version + 1;
```

Table 3.1: Bulk update time

DB_A time (ms)	DB_B time (ms)	DB_C time (ms)	DB_D time (ms)
26	13	58	9

The bulk alternative is one order of magnitude faster than batch updates. However, batch updates can benefit from application-level optimistic locking mechanisms, which are suitable for preventing *lost updates* when data is loaded in a read-only database transaction and written back in a successive transaction.

Like with updates, bulk deleting is also much faster than deleting in batches.

```
DELETE FROM post_comment WHERE version > 0;
DELETE FROM post WHERE version > 0;
```

Table 3.2: Bulk delete time

DB_A time (ms)	DB_B time (ms)	DB_C time (ms)	DB_D time (ms)
3	12	1	2

Long-running transaction caveats

Processing too much data in a single transaction can degrade application performance, especially in a highly concurrent environment. Whether using 2PL (Two-Phase Locking) or MVCC (Multiversion Concurrency Control), writers always block other conflicting writers.

Long running transactions can affect both batch updates and bulk operations if the current transaction modifies a very large number of records. For this reason, it is more practical to break a large batch processing task into smaller manageable ones that can release locks in a timely fashion.

3.3 Retrieving auto-generated keys

It is common practice to delegate the row identifier generation to the database system. This way, the developer does not have to provide a monotonically incrementing primary key since the database takes care of this upon inserting a new record.

As convenient as this practice may be, it is important to know that auto-generated database identifiers might conflict with the batch insert process.

Like many other database features, setting the auto incremented identifier strategy is database-specific so the choice goes between an *identity* column or a database *sequence* generator.

Oracle

Prior to Oracle 12c, an auto incremented generator had to be implemented on top of a database sequence.

```
CREATE SEQUENCE post_seq;

CREATE TABLE post (
    id NUMBER(19,0) NOT NULL,
    title VARCHAR2(255 CHAR),
    version NUMBER(10,0) NOT NULL,
    PRIMARY KEY (id));

CREATE OR REPLACE TRIGGER post_identity
BEFORE INSERT ON post
FOR EACH ROW
BEGIN
    SELECT post_seq.NEXTVAL
    INTO   :NEW.id
    FROM   dual;
end;
```

Oracle 12c adds support for identity columns as well, so the previous example can be simplified as follows.

```
CREATE TABLE post (
    id NUMBER(19,0) NOT NULL GENERATED ALWAYS AS IDENTITY,
    title VARCHAR2(255 CHAR),
    version NUMBER(10,0) NOT NULL,
    PRIMARY KEY (id));
```

SQL Server

Traditionally, SQL Server offered identity column generators, but, since SQL Server 2012, it now supports database sequences as well.

```
CREATE TABLE post (
    id BIGINT IDENTITY NOT NULL,
    title VARCHAR(255),
    version INT NOT NULL,
    PRIMARY KEY (id));
```

PostgreSQL

PostgreSQL 9.5 does not support identity columns natively, although it offers the `SERIAL` column type which can emulate an identity column.

```
CREATE TABLE post (
    id SERIAL NOT NULL,
    title VARCHAR(255),
    version INT4 NOT NULL,
    PRIMARY KEY (id));
```

The `SERIAL` (4 bytes) and `BIGSERIAL` (8 bytes) types are just a syntactic sugar expression as, behind the scenes, PostgreSQL relies on a database sequence anyway.

The previous definition is therefore equivalent to:

```
CREATE SEQUENCE post_id_seq;

CREATE TABLE post (
    id INTEGER DEFAULT NEXTVAL('post_id_seq') NOT NULL,
    title VARCHAR(255),
    version INT4 NOT NULL,
    PRIMARY KEY (id));
);
```